Sila (صِلة)

Assignment 1

Course: AI1220 — Software, Web & Mobile Engineering

Project Overview

Sila (صِلة) is an Arabic word meaning *connection* or *bond*. The name represents the purpose of our platform: bringing university students together and strengthening connections within the campus community.

Sila is a private social network designed for students within one university. Verified students can create and join communities, share posts, interact through likes and comments, and collaborate on event announcements. The platform also includes audience controls, moderation, and AI-powered discussion summaries.

The aim is to provide a safe and organized space where students can communicate, discover campus activities, and participate in university life.

---

1 Requirements Engineering

1.1 Stakeholder Analysis

The following stakeholders have been identified for Sila.

Stakeholder	Goals	Concerns	Influence
SH-01 Students	Share posts, join communities, and interact with others	Privacy, inappropriate content, and account security	High
SH-02 Community Organizers	Manage communities and collaborate on announcements	Unauthorized edits and low participation	High

SH-03 Moderators	Review reports and enforce community rules	False reports and unfair decisions	High
SH-04 University Administrators	Approve official communities and oversee university policies	Misinformation and policy violations	High
SH-05 Development/IT Team	Build and maintain a secure and reliable platform	Security, performance, and maintenance	Medium

Stakeholder Interests and Potential Conflicts

Different stakeholder groups may have competing interests:

- **Students and moderators:** Students want freedom to share opinions, while moderators need to maintain a safe environment. Clear community rules and an appeal process help balance these interests.
- **Students and university administrators:** Students value privacy, while administrators need to oversee university policies. Access permissions limit unnecessary exposure of student content.
- **Community organizers and members:** Organizers need control over community activities, while members want to participate. Role-based permissions define who can edit shared content.
- **Moderators and administrators:** Moderators handle individual reports, while administrators oversee policy enforcement and escalated appeals.

These are potential differences in stakeholder interests, not disagreements within our project team.

1.2 Functional Requirements

The functional requirements describe what Sila must allow users to do. Each requirement is written so it can be tested.

FR-01 — Identity and Account Verification

The system shall allow students to register using an approved university email domain, verify their email address, log in, and log out.

- Students must verify their email before accessing the platform.
- Registration using an unapproved email domain shall be rejected.

- Invalid login credentials shall be rejected.

FR-02 — Community Creation and Membership

The system shall allow verified students to create and join university communities.

- Students can create communities without administrator approval.
- University administrators must approve a community before it receives official university status.
- Students can join open communities immediately.
- Restricted communities require membership approval from an authorized organizer.
- Users without permission cannot access restricted community content.

FR-03 — Publishing Posts

The system shall allow authenticated users to create, view, edit, and delete posts according to their permissions.

- Posts may contain rich text.
- Posts must not exceed **2,000 characters**.
- Empty posts and posts exceeding the character limit shall be rejected.
- Users cannot edit or delete posts they are not authorized to manage.

FR-04 — Real-Time Collaboration

The system shall allow authorized users to collaborate on shared event and announcement drafts.

- Authorized collaborators can edit shared drafts.
- Changes shall be shown to other active collaborators in real time.
- Unauthorized users cannot edit shared drafts.
- The system shall detect conflicting or outdated edits and prevent them from silently overwriting newer changes.
- Reconnected collaborators shall be able to retrieve the latest saved version.

FR-05 — Audience Control and Privacy

The system shall allow post authors to select the audience for their posts.

- Authors can make posts visible to all verified students or only to members of a selected community.
- The system shall check access permissions before displaying restricted posts.
- Users outside the selected audience cannot view the post.

FR-06 — Likes and Comments

The system shall allow authorized users to like, unlike, and comment on posts they can access.

- Each user can have only one active like per post.
- Users can remove their likes.
- Comments must not be empty.
- Comments must not exceed **500 characters**.
- Users cannot interact with posts they are not authorized to access.

FR-07 — Reporting, Moderation, and Appeals

The system shall allow users to report inappropriate content and authorized moderators to review reports.

- Verified users can submit content reports.
- Dedicated moderators can review reports and dismiss them or take moderation action.
- Moderation decisions shall be recorded.
- Affected users can submit an appeal within **seven days** of a moderation decision.
- Appeals submitted after seven days shall be rejected.
- University administrators shall review escalated appeals.
- Users without moderation permissions cannot perform moderation actions.

FR-08 — AI Discussion Summarization

The system shall allow authorized users to request AI-generated summaries of discussions they can access.

- A summary is generated only when the user selects **Summarize**.
- The AI shall use only discussion content the requesting user is authorized to access.
- The generated result shall be clearly labeled as an AI-generated summary.
- The system shall verify that the result is nonempty and associated with the requested discussion.
- Users shall be able to view the original discussion to check the summary.
- If summarization fails, the system shall display an error message without changing or deleting the original discussion.

AI requirement details

- **User need:** Students need a quicker way to understand long discussion threads.
- **Input data:** Text from discussion posts and comments that the requesting user is allowed to view.
- **Output:** A short, clearly labeled summary of the selected discussion.
- **Verification:** The system checks the response structure and discussion association. Users can compare the summary with the original content to check accuracy. The system does not guarantee that every AI-generated statement is correct.

1.3 Non-Functional Requirements

The non-functional requirements define the quality standards Sila should meet to provide a secure, reliable, and user-friendly platform. Each requirement includes a measurable target and a justification.

NFR-01 — Performance

The system shall respond to at least 95% of post creation and retrieval requests within 3 seconds under a test load of 50 concurrent users.

Justification: Students should be able to browse and publish posts without unnecessary delays.

NFR-02 — Scalability

The system shall support at least 500 registered users and handle 50 concurrent users while meeting the performance target.

Justification: The platform should support increasing participation as more students join.

NFR-03 — Availability

The system shall target 99% monthly availability, excluding scheduled maintenance.

Justification: Students need reliable access to campus discussions and announcements.

NFR-04 — Recovery

The system shall back up stored data at least once every 24 hours and aim to restore service within 4 hours after a recoverable system failure.

Justification: Posts, community information, and announcements should be protected against data loss.

NFR-05 — Security

The system shall securely hash stored passwords and reject unauthorized access to protected functions. Production communication shall use HTTPS.

Justification: Student accounts and platform functions must be protected against unauthorized access.

NFR-06 — Privacy

The system shall restrict community-only posts to authorized members and prevent unauthorized discussion content from being sent to the AI summarization service.

Justification: Students should have control over who can access their content.

NFR-07 — Usability

At least 4 out of 5 first-time test users shall be able to create a post and select its audience without assistance.

Justification: Students should be able to use the main features without training.

NFR-08 — Accessibility

The system shall support keyboard navigation for its main functions and meet WCAG 2.2 Level AA requirements for text contrast.

Justification: The platform should be accessible to students with different needs.

NFR-09 — Auditability

The system shall record moderation decisions with the moderator ID, action taken, affected content ID, and timestamp.

Justification: Moderation decisions must be traceable to support accountability and appeals.

NFR-10 — Real-Time Collaboration

Under a test load of 10 active collaborators, at least 95% of successfully saved draft updates shall appear to other connected collaborators within 2 seconds.

Justification: Community organizers need timely updates when working together on announcements.

Verification of Non-Functional Requirements

This will be verified through performance testing, access-control testing, usability testing, accessibility checks, and review of system logs. The proposed targets will be evaluated using the available testing environment during Assignment 2.

1.4 User Stories and Acceptance Criteria

US-01 — Student Publishing a Post

As a student, I want to create a post and select who can see it so that I can share information with the appropriate audience.

Related requirements: FR-03, FR-05

Acceptance criteria:

1. Given a verified student enters valid post content within 2,000 characters and selects an allowed audience, when the student publishes the post, then the system saves it and displays it to the selected audience.
2. Given the student submits an empty post or exceeds 2,000 characters, when they try to publish, then the system rejects the post and displays an error.
3. Given a user is outside the selected audience, when they try to access the post, then access is denied.

US-02 — Organizer Collaborating on an Announcement

As a community organizer, I want to edit an announcement with other authorized collaborators so that we can prepare community events together.

Related requirement: FR-04

Acceptance criteria:

1. Given two authorized collaborators are editing a shared draft, when one saves a change, then the other receives the updated content in real time.
2. Given an unauthorized user attempts to edit the draft, when the request is submitted, then the system denies the action.
3. Given a collaborator submits an outdated edit that conflicts with a newer version, when the system processes it, then the conflict is detected and the newer content is not silently overwritten.
4. Given a collaborator reconnects after losing connection, when the draft is reopened, then the latest saved version is retrieved.

US-03 — Student Requesting an AI Summary

As a student, I want to summarize a long discussion so that I can understand its main points more quickly.

Related requirement: FR-08

Acceptance criteria:

1. Given a student can access a discussion, when they select Summarize, then the system generates and displays a clearly labeled AI summary.
2. Given the summary is displayed, when the student wants to check its accuracy, then the original discussion remains available.
3. Given the student does not have access to a discussion, when they request a summary, then the system denies the request.
4. Given the AI service fails, when summarization is attempted, then an error is displayed and the original discussion remains unchanged.

US-04 — Moderator Reviewing a Report

As a moderator, I want to review reported content so that I can help maintain a safe university community.

Related requirement: FR-07

Acceptance criteria:

1. Given a moderator opens a submitted report, when they review the content, then they can dismiss the report or take an authorized moderation action.
2. Given a moderator makes a decision, when the action is confirmed, then the decision is recorded.
3. Given a user without moderation permissions attempts to resolve a report, when the action is submitted, then access is denied.
4. Given an affected user appeals within seven days, when the appeal is submitted, then the system accepts it for review.
5. Given an appeal is submitted after seven days, when the system checks the deadline, then the appeal is rejected.
6. Given an appeal is escalated, when it reaches the appropriate stage, then an authorized university administrator can review it.

1.5 Requirements Validation and Traceability

Requirements Review

The draft was checked for coverage, testability, and consistency. The review identified unclear rules concerning university configuration, restricted-community audiences, collaboration permissions, and appeal escalation. The following policies clarify these areas.

Policies and Assumptions

- Each deployment serves one university with configurable approved email domains.
- Different universities use separate deployments and databases.
- Only verified university users access the platform.
- Open communities allow immediate joining; restricted communities require organizer approval.
- Restricted-community posts remain visible only to approved members.
- Posts outside restricted communities may target all verified students or members of a selected community.
- Only approved organizers of the relevant community edit and publish shared announcements.
- Outdated draft saves are rejected without overwriting newer content.
- Administrators approve official-community requests submitted by organizers.
- Affected authors may appeal within seven days.
- A different moderator reviews an appeal. It escalates when independent review is unavailable or university policy is involved.
- AI summarizes authorized published discussions on request. Drafts and hidden content are excluded.

Open Questions

Question	Owner
Which university domains will be accepted?	University administrator
Is external AI processing permitted?	University policy owner
Which hosting provider, email provider, and permitted AI provider will be used? (The main technologies are chosen in Section 2.2.)	Coder, with university approval
How will monitoring, backups, and restoration be set up?	Coder or deployment owner

Architecture Identifiers

ID	Component
WEB	React frontend
API	FastAPI backend
DB	PostgreSQL database
C1	HTTP and WebSocket Gateway
C2	Identity and Access Control
C3	Community Management
C4	Publishing and Interaction
C5	Draft Collaboration
C6	Moderation and Appeals
C7	AI Integration
OPS	Configuration, monitoring, and backup procedures

Traceability Matrix

Stakeholder Need	Requirements	Story / Scenario	Components
SH-01: Secure access	FR-01, NFR-05	SC-01: Registration and login	WEB, C1, C2, DB
SH-01, SH-02: Community participation	FR-02	SC-02: Joining and approval	WEB, C2, C3, DB
SH-01: Publishing and privacy	FR-03, FR-05, NFR-06	US-01	WEB, C2, C4, DB
SH-02: Shared announcements	FR-04, NFR-10	US-02	WEB, C1, C2, C5, DB
SH-01: Likes and comments	FR-06	SC-03: Post interaction	WEB, C2, C4, DB
SH-03, SH-04: Moderation and appeals	FR-07, NFR-09	US-04	WEB, C2, C6, DB

SH-04: Official communities	FR-02	SC-04: Official approval	WEB, C2, C3, DB
SH-01: Discussion summaries	FR-08, NFR-06	US-03	WEB, C2, C7, DB
SH-01, SH-05: Responsive service	NFR-01, NFR-02	SC-05: Load testing	WEB, API, DB
SH-05: Reliable service	NFR-03, NFR-04	SC-06: Monitoring and restoration	API, DB, OPS
SH-01: Usability and accessibility	NFR-07, NFR-08	SC-07: Interface testing	WEB

Additional Scenarios

- **SC-01:** Reject unapproved domains, unverified access, and invalid credentials.
- **SC-02:** Allow open joining and require approval for restricted joining.
- **SC-03:** Enforce one active like per user per post and validate comments.
- **SC-04:** Allow only administrators to approve official status.
- **SC-05–SC-07:** Perform the corresponding checks in Section 1.3.

Scope Clarification

Multiple universities sharing one deployment are outside scope. The existing feature exclusions remain unchanged.

---

2 System Architecture

2.1 Architectural Drivers

The following requirements have the greatest influence on Sila's design.

Rank	Driver	Design Response
1	Security and privacy	Check identity, roles, ownership, membership, and audience in the backend.
2	Safe collaboration	Use draft-version checks to prevent silent overwrites and WebSockets to distribute saved updates.

3	Moderation accountability	Store decisions, appeals, and audit records.
4	Required technologies and manageable development	Use React, FastAPI, WebSockets, and an LLM integration within one modular backend.
5	Performance and reliability	Use indexed queries, testing, monitoring, backups, and restoration procedures.

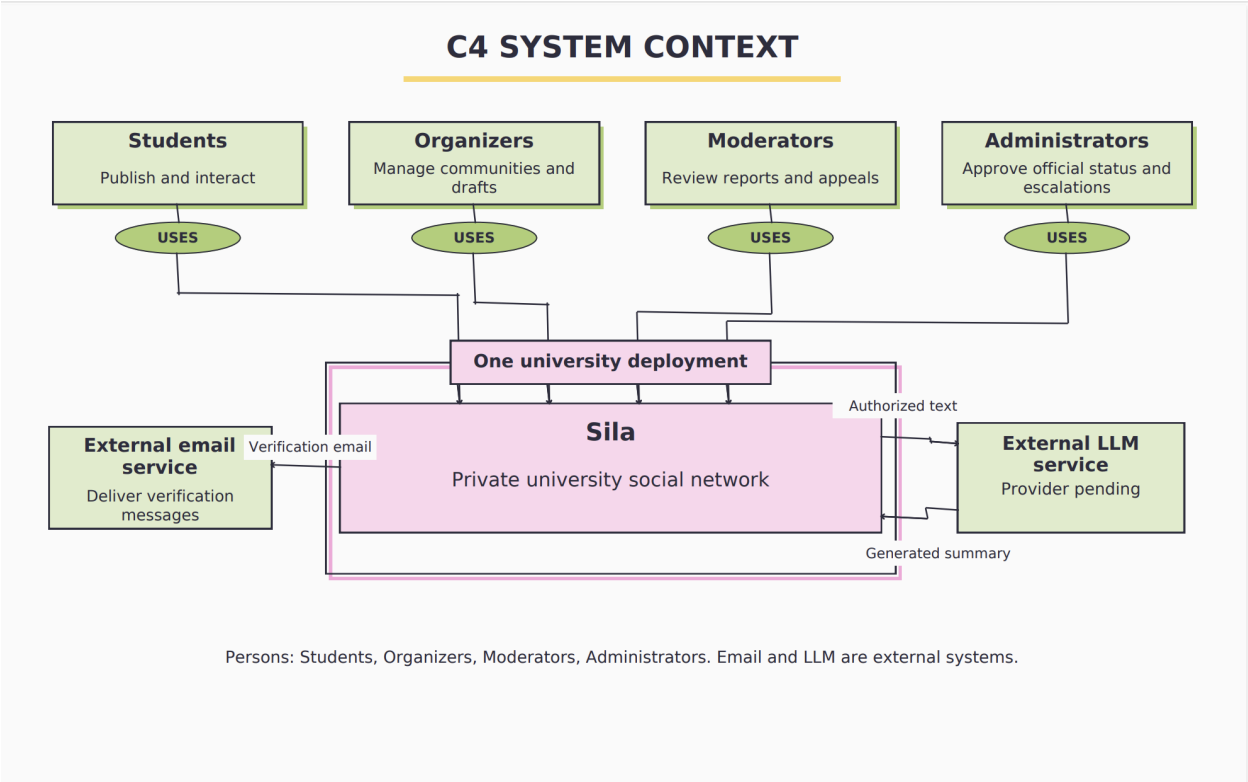
One backend divided into modules keeps development and deployment manageable. PostgreSQL is proposed because Sila's records have relationships and draft saves require reliable transactions.

2.2 System Design

The planned Assignment 2 system uses React, FastAPI, and PostgreSQL. This design is separate from the smaller Assignment 1 PoC and remains subject to team review.

2.2.1 Context and containers

The context diagram shows students, organizers, moderators, and administrators using Sila at one university. Email verification and AI summaries use external services outside Sila's boundary.



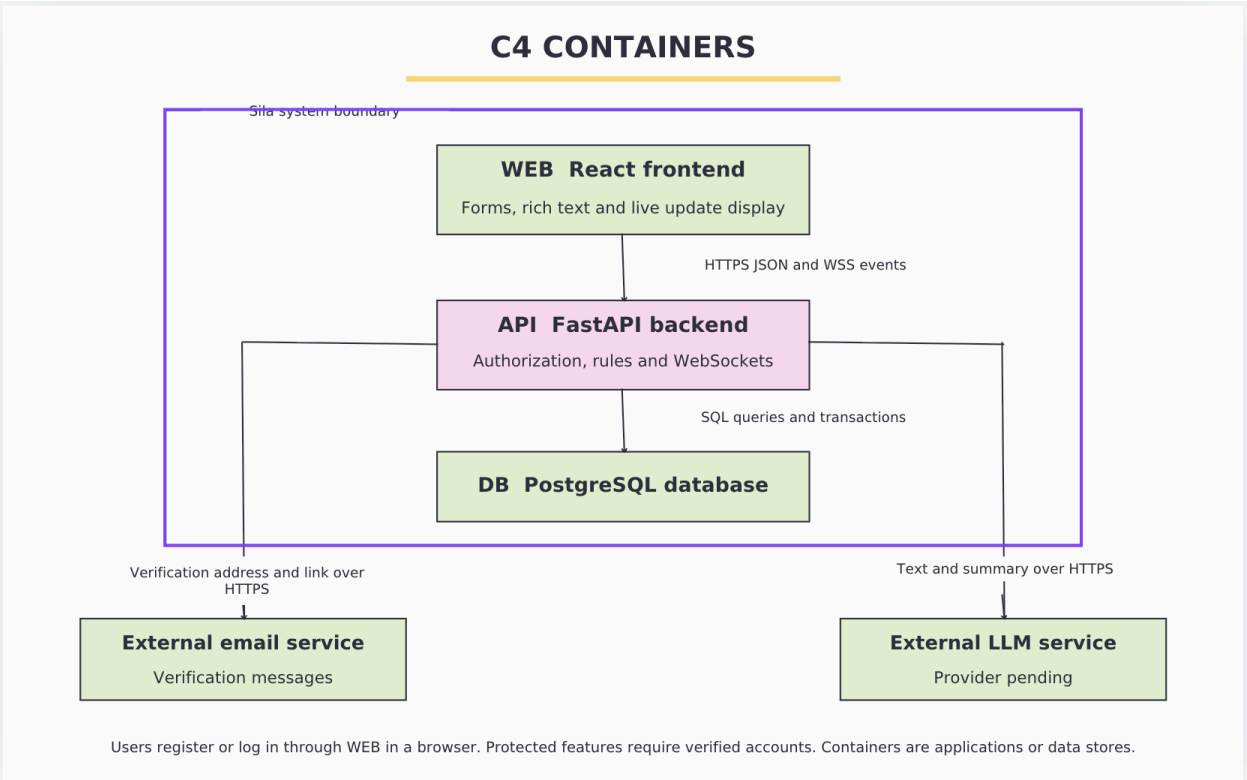
https://canva.link/mnbu5ng6gg0l7hs

The container diagram shows three parts inside that boundary:

Part	Technology and purpose
WEB	React: pages, forms, rich-text editing, and display of live updates
API	Python/FastAPI: requests, permissions, business rules, and WebSocket connections
DB	PostgreSQL: linked records such as users, posts, and communities

React sends requests to FastAPI over HTTPS. WebSockets carry saved draft updates. FastAPI accesses PostgreSQL and calls the email and AI services; the browser does not access them directly.

React, rich text, FastAPI, WebSockets, and an LLM meet Assignment 2's technology constraints. PostgreSQL fits the linked data and lets related changes succeed or fail together.



https://canva.link/mnbu5ng6gg0l7hs

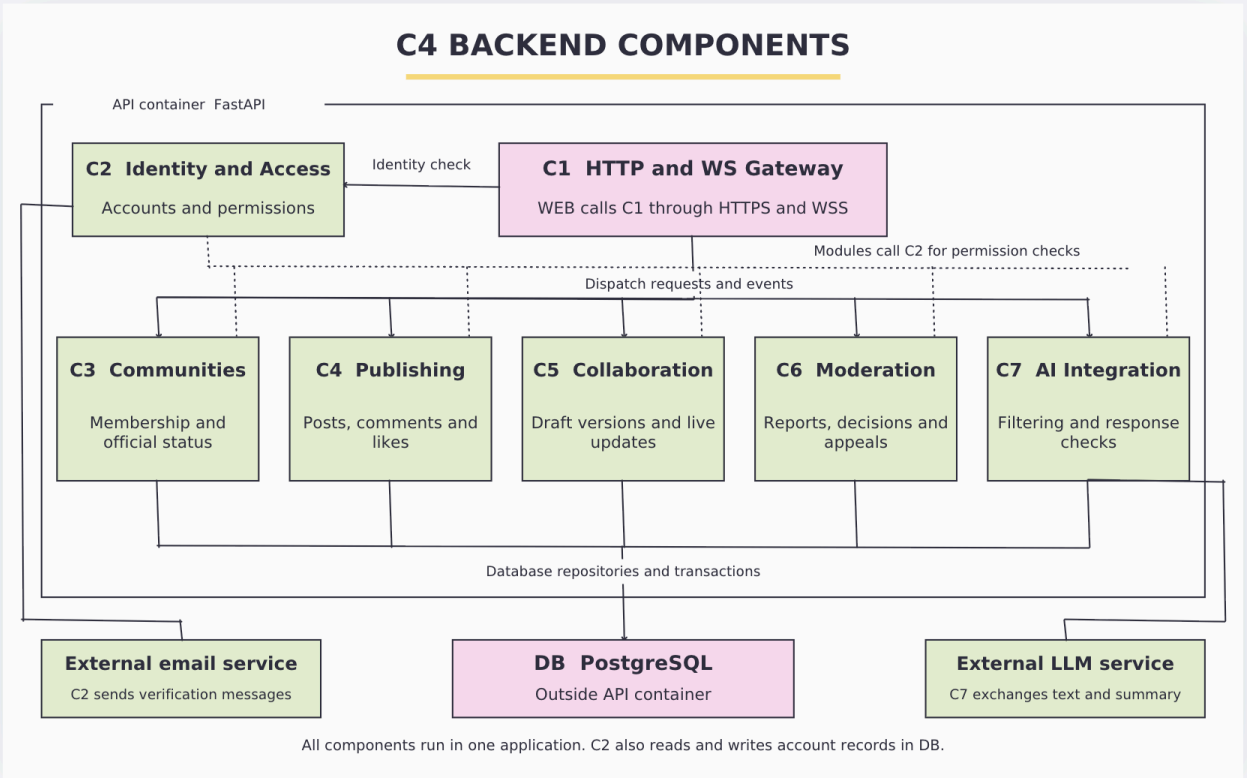
2.2.2 Backend components

The component diagram looks inside API. C1–C7 are modules in one backend, not separate applications.

Component	Responsibility
C1: HTTP and WebSocket Gateway	Receive and route requests and events
C2: Identity and Access Control	Check accounts, roles, ownership, and audiences

C3: Community Management	Manage communities, membership, and official status
C4: Publishing and Interaction	Validate and manage posts, comments, and likes
C5: Draft Collaboration	Check draft versions, save, publish, and send updates
C6: Moderation and Appeals	Handle reports, decisions, appeals, and their records
C7: AI Integration	Send permitted content and check summary responses

One backend keeps shared permission rules and database changes together.



https://canva.link/mnbu5ng6gg0l7hs

2.2.3 Authentication and rich text

Users verify an approved university email address. Passwords are stored as hashes, not readable passwords. A cookie identifies the login session; logout ends it. The backend checks permissions before protected actions and updates.

The server sanitizes rich text by removing unsafe markup before storing it. Posts allow 1–2,000 visible characters; plain-text comments allow 1–500. Empty drafts may be saved but cannot be published. These limits count Unicode code points after trimming outer whitespace.

2.2.4 Deployment and external services

The plan uses one server running one FastAPI process so WebSocket updates share memory; more processes would need shared messaging. OPS covers health checks, daily database backups, and restore procedures, supporting NFR-03 and NFR-04.

Email verifies accounts. AI summarizes only content the requester may access, and its result is labelled AI-generated. Failure leaves the discussion unchanged. Secrets stay in backend environment variables.

Hosting and providers have not been set up. External AI processing needs university permission. Development can use mocks, meaning simulated service responses, without paid calls.

2.3 Behaviour and Design Decisions

Audience and Authorization

C2 checks permissions before protected requests, draft subscriptions, saves, and AI requests.

- Authors manage their own ordinary posts.
- Restricted-community posts remain visible only to approved members.
- Only approved community organizers edit and publish shared announcements.
- Moderators and administrators access only the evidence needed for their assigned reviews.

C4 sanitizes rich text and rejects empty content. Limits apply to visible text: 2,000 characters per post and 500 per comment.

Publishing and Collaboration

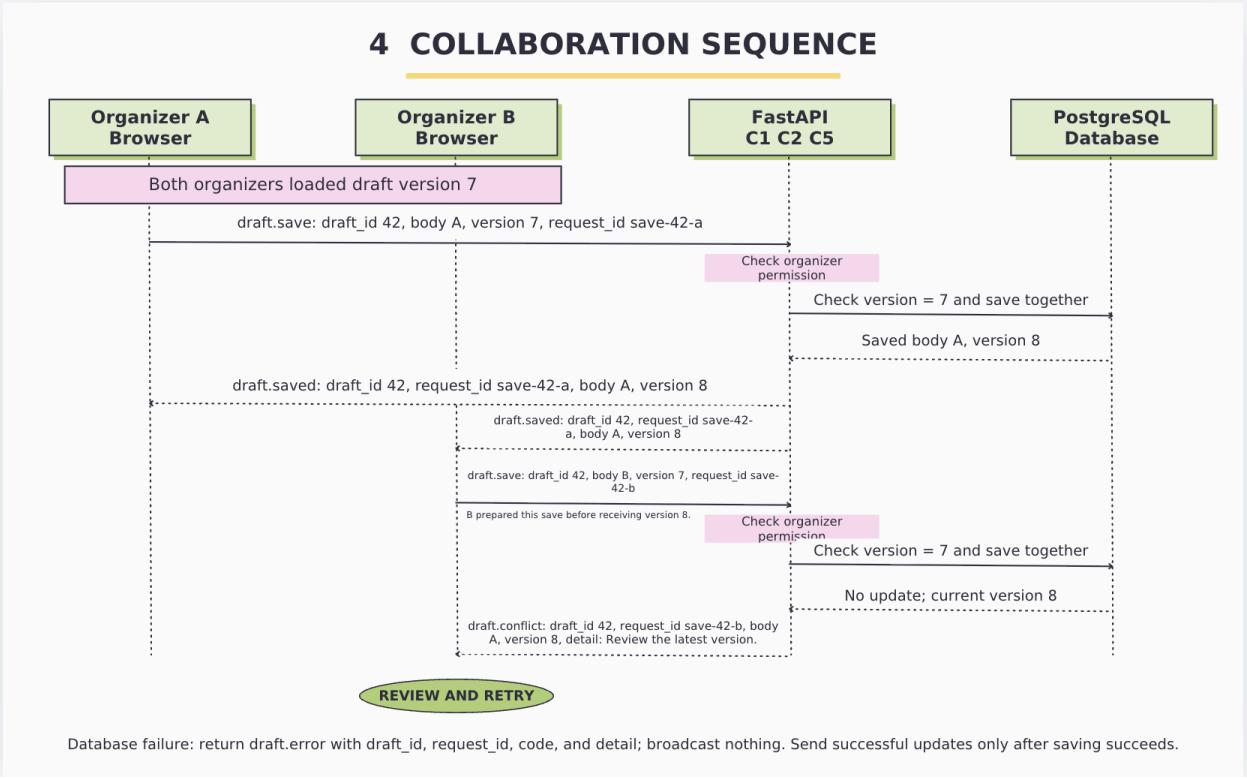
C5 checks the submitted draft version and updates it in one database transaction. Successful saves increase the version and are broadcast after the transaction commits.

Outdated saves are rejected. Organizers review the latest content and manually resolve conflicts. After reconnecting, they retrieve the latest saved draft.

Publishing rechecks organizer permissions, creates the post, and marks the draft published in one transaction. Repeated requests must not create duplicate posts.

Sequence Diagram

The diagram shows two organizers who loaded version 7. The first save creates version 8. The second save still references version 7 and is rejected without overwriting the newer content.



https://canva.link/mnbu5ng6gg0l7hs

Failure Handling

Situation	System Response
Outdated draft save	Reject the save and provide the latest version.
Lost connection	Reauthorize and retrieve the latest draft.
Database save failure	Return an error and broadcast no successful update.
Lost save acknowledgement	Retrieve the current snapshot before retrying.
Permission removed	Reject further actions and stop protected updates.
AI timeout or invalid response	Return an error without changing the discussion.

Moderation and Appeals

C6 allows moderators to dismiss reports or hide reported content. Decisions record the actor, action, target, reason, and timestamp.

Affected authors may appeal within seven days. A different moderator reviews the appeal. It escalates to an administrator when independent review is unavailable or university policy is involved.

Hidden content remains review evidence but is excluded from normal access and AI. Appeal resolutions are recorded. Reversing moderation does not restore content independently deleted by its author.

AI Summarization

C7 collects only the selected discussion content the requester may access. It excludes drafts, hidden content, unrelated discussions, and account metadata.

External processing requires university permission. Results are labeled AI-generated and checked for nonempty content and association with the request. The original discussion remains accessible.

These checks do not guarantee accuracy. Personal information entered in discussion text may still be present.

Development and Testing

Test permissions, draft-version checks, appeal deadlines, and AI filtering separately. Then test browsers, API, and DB together.

Use browser sessions to test conflicting saves, reconnection, and permission changes. Mock email and AI services to test failures without paid calls.

2.4 Interfaces

Section 2.4.1 describes the implemented PoC. The remaining interfaces are planned for Assignment 2.

2.4.1 Implemented proof-of-concept interface

FastAPI serves the page at `GET /` and the API from the same local address. The browser exchanges JSON with the backend. Both post routes need no login and use fictional content. The contract is `docs/contracts/poc-posts.md`.

- `POST /posts` requires `application/json` containing a JSON object with a string `body`. Success returns `201` with integer `post_id` and string `body`. Invalid input returns `422` with a string `detail`.
- `GET /posts` takes no request body and returns `200` with a JSON array, ordered by descending `post_id`. An empty feed returns `[]`.

Example creation on a fresh backend:

```
{"body":"  Club meeting today.  "}
```

```
{"post_id":1,"body":"Club meeting today."}
```

Retrieving that feed returns `[{"post_id":1,"body":"Club meeting today."}]`.

The backend rejects non-string values without conversion, trims outer whitespace, and accepts 1–2,000 Unicode code points. Internal spaces and line breaks remain. Stored and returned text is trimmed. IDs start at 1 and increase only after successful creation. Rejected requests create nothing and consume no ID.

Every documented validation failure returns `422` with one exact `detail` message:

Invalid input	**Exact message**
Malformed JSON	`Request body must be valid JSON.`
JSON that is not an object	`Request body must be a JSON object.`
Missing `body`	`The body field is required.`

Non-string `body`, including `null`	`Post body must be a string.`
Empty, whitespace-only, or over-limit text	`Post body must contain 1–2000 characters after trimming.`

Posts remain in memory after browser refresh. Restarting the backend clears posts and resets IDs. The backend's count decides acceptance; the browser counter also counts code points. HTML-like input is displayed literally using `textContent`.

The page loads the feed on opening. Publish is disabled while saving. Success clears the input and reloads the feed; validation failure keeps the input and shows the error. A network failure means the result is unconfirmed, so the page does not automatically retry. It reports separately when creation succeeds but feed refresh fails.

2.4.2 Proposed Assignment 2 HTTP interfaces

All routes below start with `/api/v1` and use HTTPS and JSON. A login cookie identifies the user. C2 checks verification, ownership, roles, and audience; sending a user ID cannot grant permission.

Success uses `200` for reads/updates, `201` for creation, and `204` for deletion with no response body. Errors contain a string `detail`: `401` means login is missing or invalid; `403`, permission denied; `404`, unavailable or hidden resource; `409`, conflicting state; `422`, invalid input; `500`, unexpected server failure; `502`, failed or invalid provider response; `503`, service unavailable; and `504`, provider timeout. Errors do not expose secrets.

Route, after `/api/v1`	**Main fields exchanged**	**Who is allowed; main failures**
`POST /accounts`, `POST /accounts/verify`, `POST /sessions`, `DELETE /sessions/current`	`email`, `password`, or verification `token`; return verification status or `user_id` and roles	Registration/login need no session; logout needs one. Approved university email required. `401`, `422`.

`POST /communities`, `POST /communities/{community_id}/memberships`	`name`, `visibility`; return `community_id` or membership `status`	Verified users. Restricted joins await organizer approval. `403`, `422`.
`POST /communities/{community_id}/official-requests`, `PATCH /official-requests/{request_id}`	Request `reason`; decision `status`; return `request_id`, `community_id`, `status`	Organizers request; administrators decide. `403`, `409`, `422`.
`GET /posts`, `POST /posts`, `PATCH /posts/{post_id}`, `DELETE /posts/{post_id}`	`body`, `audience`, optional `community_id`; returned posts include `post_id`, `author_id`	Verified users; viewers must be in the audience, authors edit/delete. Restricted-community posts stay restricted. `403`, `404`, `422`.
`PUT /posts/{post_id}/likes/me`, `DELETE /posts/{post_id}/likes/me`, `POST /posts/{post_id}/comments`	Like result: `post_id`, `liked`; comment: `body` → `comment_id`, `post_id`, `author_id`, `body`	Permitted viewers. One like per user/post; comments 1–500 characters. `403`, `404`, `422`.

`POST /reports`, `POST /reports/{report_id}/decisions`	`target_type`, `target_id`, `reason`; decision `action`, `reason`; return `report_id`, `status` or `decision_id`, `action`, `decided_at`	Users report accessible content; authorized moderators decide. `403`, `404`, `409`, `422`.
`POST /decisions/{decision_id}/appeals`, `PATCH /appeals/{appeal_id}`	`reason`; resolution `outcome`, `reason`; return `appeal_id`, `status`, `submitted_at` or outcome/time	Affected author appeals within seven days. A different moderator reviews; escalations need an administrator. `403`, `422`.
`GET /drafts/{draft_id}`, `POST /drafts/{draft_id}/publish`	Read: `draft_id`, `body`, `version`; publish: `request_id`, `version` → `draft_id`, `post_id`, `status`	Approved organizers of that community. `403`, `404`, `409`, `422`.
`POST /posts/{post_id}/summary`	Return `post_id`, `summary`, `ai_generated: true`	Permitted viewers; backend selects the discussion text. `403`, `404`, `502`, `503`, `504`.

Post content is sanitized before its 2,000-character limit is checked. Only permitted content is returned.

2.4.3 Proposed collaboration interface

Organizers connect to `/api/v1/drafts/{draft_id}/events` over WebSocket. Permissions are checked before saves and protected updates. A save sends the version the organizer loaded:

```
{"type":"draft.save","draft_id":42,"request_id":"save-42-a","version":7,"body":
"<p>Club meeting at 4 pm.</p>"}
```

The backend checks the version and saves together. Success sends `draft.saved` with `draft_id`, `request_id`, saved `body`, and the new `version` (8 here), then broadcasts the saved update. An outdated save gets `draft.conflict` with those identifiers, the latest body/version, and an explanatory `detail`. Organizers resolve the conflict manually.

Other failures send `draft.error` with `draft_id`, `request_id`, `code`, and `detail`. Denied users receive no draft content. A failed save broadcasts nothing. These are WebSocket messages, not HTTP status codes.

Each save has a request ID so a retried save is not applied twice. Reconnecting reloads the latest draft after checking permission.

Publishing saves the post and its draft link together. Draft's unique `published_post_id` identifies the published result. First publication returns `201`; repeating it returns that same post with `200`. An outdated unpublished draft returns `409`.

2.4.4 Proposed external service interfaces

The backend sends the email service a university address and verification link, and receives acceptance or failure. Acceptance does not verify the account; the user must follow a valid link. Failed delivery can be retried.

For AI, the backend sends only authorized published discussion text, excluding drafts and hidden/deleted content. It checks that the response is nonempty and belongs to the requested discussion, then labels it AI-generated. Failure leaves the discussion unchanged. University permission is required before external processing. These services are planned, not connected in the PoC.

2.5 Code and Repository Structure

The current repository contains the Assignment 1 PoC. Assignment 2 folders below are planned, not implemented.

Current location	Contents

`poc/app/main.py`	FastAPI routes, validation, and in-memory posts
`poc/app/index.html`	Browser page, styling, and JavaScript
`poc/tests/`	Contract tests
`poc/requirements.txt`, `poc/README.md`	Dependencies, setup, tests, and demo instructions
`docs/contracts/`, `docs/report/`	PoC contract and report sections
`docs/diagrams/`	Mermaid sources and Canva change checklist
`.github/workflows/poc-tests.yml`	Automated PoC checks on GitHub
`CLAUDE.md`, `.gitignore`	AI workflow instructions and files excluded from Git

Planned location	**Responsibility**
`frontend/`	React pages, rich-text editor, and WebSocket client
`backend/app/api/`	C1: HTTP routes and WebSocket messages

backend/app/services/	C2–C7: permission checks and business rules
backend/app/models/, backend/app/repositories/	Database records and access
backend/app/integrations/	Email and AI service calls
backend/app/core/	Shared settings and login helpers
backend/tests/, frontend/tests/	Backend, browser, and integration tests
config/, .env.example	Configuration examples with placeholders only

This separates pages, business rules, and data access so each can be tested. Assignment 2 will sit alongside poc/, keeping the demonstration reproducible. Mohammed is responsible for the current PoC under the recorded team allocation; Assignment 2 ownership must be agreed in Section 3.1.

Secrets belong only in backend environment variables, never in Git or browser code. Local environments and personal settings are excluded from Git. Existing documentation folders will also hold Assignment 2 material.

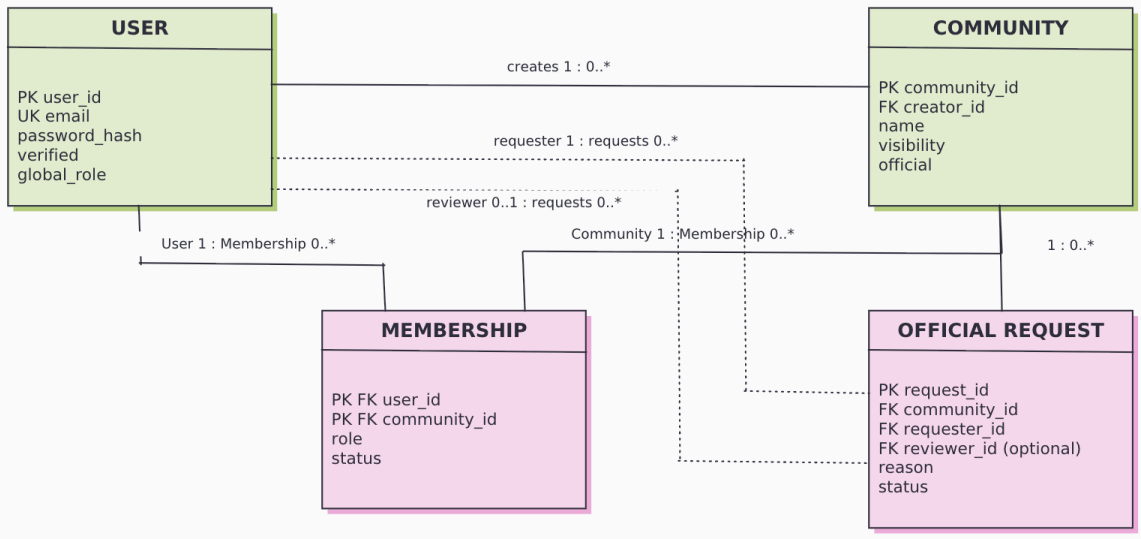
2.6 Data Model

The PoC stores only integer `post_id` and string `body` in memory. The ER diagram describes the planned PostgreSQL model for Assignment 2, not an existing database.

2.6.1 Entities and relationships

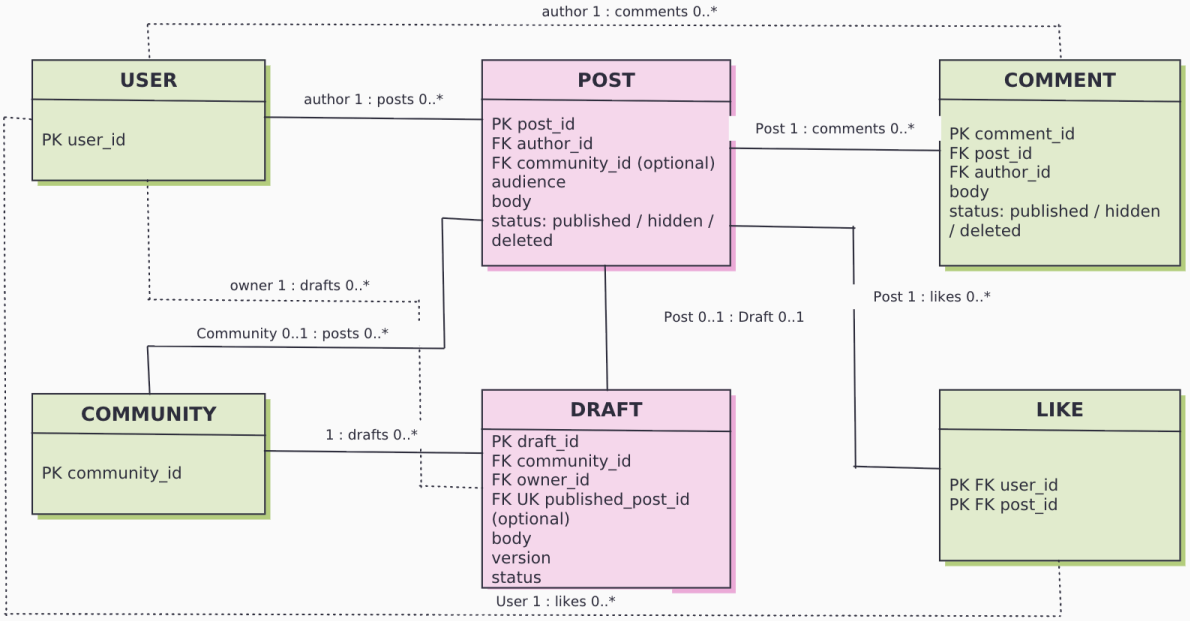
Each entity has an ID. References link related records; for example, author_id links a post to a user. Membership and Like identify their records using a pair of IDs.

5A DATABASE MODEL USERS AND COMMUNITIES



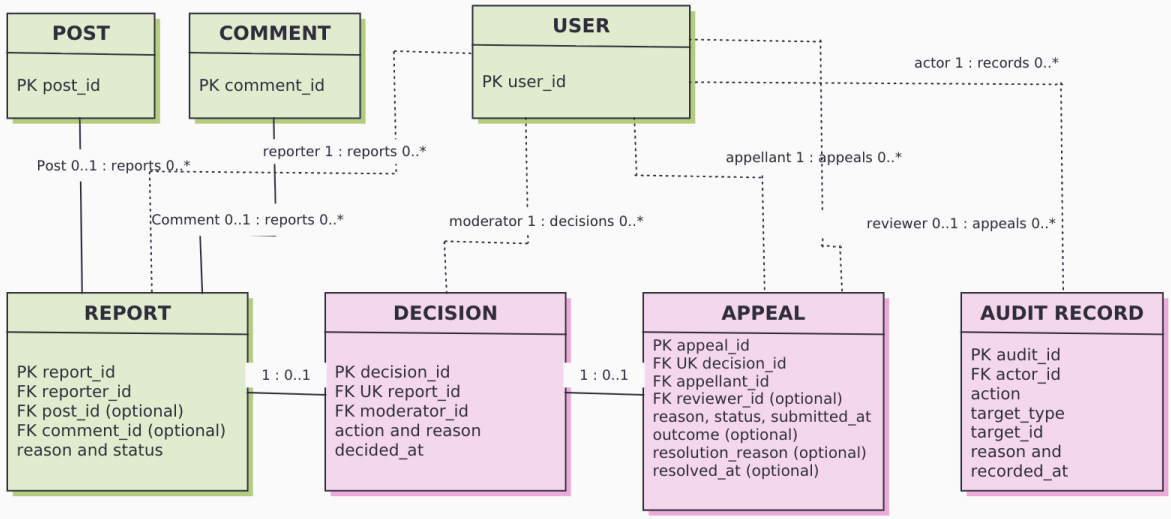
PK primary key FK foreign key UK unique. Dashed lines are role-specific User relationships.

5B DATABASE MODEL PUBLISHING



Each user has at most one active like per post. Community and User repeat the same entities from 5A.
Reversing moderation changes hidden to published only; never restore deleted content. Keep hidden if another active decision requires it.
Check the draft and save the post and unique published_post_id together. Repeated publication returns the linked post.

5C DATABASE MODEL MODERATION AND APPEALS



Exactly one report target: Post OR Comment. Affected author may appeal within 7 days of decided_at. Reviewer differs from original moderator.

Post, Comment and User repeat existing entities. Ordinary users cannot modify audit records.

https://canva.link/mnbu5ng6gg0l7hs

Entity	Main information and links
User	user_id, unique email, password_hash, verified, global_role
Community	community_id, creator_id, name, visibility, official
Membership	user_id, community_id, role, status
Post	post_id, author_id, optional community_id, audience, body, status
Draft	draft_id, community_id, owner_id, body, version, status, optional unique published_post_id
Comment	comment_id, post_id, author_id, body, status
Like	user_id, post_id
OfficialRequest	request_id, community_id, requester_id, optional reviewer_id, reason, status
Report	report_id, reporter_id, one target (post_id or comment_id), reason, status
Decision	decision_id, report_id, moderator_id, action, reason, decided_at

Appeal	`appeal_id`, `decision_id`, `appellant_id`, optional `reviewer_id`, `reason`, `status`, `submitted_at`; resolution adds `outcome`, `resolution_reason`, `resolved_at`
AuditRecord	`audit_id`, `actor_id`, `action`, `target_type`, `target_id`, `reason`, `recorded_at`

Supporting account records hold login sessions and email verification information; their implementation details are outside this ER overview.

2.6.2 Constraints and policies

- Users join communities through Membership. Organizer roles belong to a community, not the whole university. Only approved organizers edit and publish its drafts.
- A user has at most one membership per community and one like per post. Removing a like removes that pair.
- A university-wide post may have no community. Community-only posts require one; restricted-community content is limited to approved members.
- Posts contain sanitized rich text, limited to 2,000 visible characters. Comments contain plain text, limited to 500. Published content cannot be empty.
- Post and Comment `status` is `published`, `hidden`, or `deleted`. Reversing moderation only changes `hidden` back to `published`; it never restores `deleted` content. Content stays hidden if another active decision still requires it.
- Draft saves check the loaded version and increase it on success. Publication creates the post and records Draft's unique `published_post_id` together, after checking the draft. Repeated publication returns the linked post instead of creating another.
- Each report targets exactly one post or comment. Each report has at most one initial decision; each decision has at most one appeal. Only the affected author may appeal within seven days of `decided_at`; a different moderator reviews it.
- Decisions and appeal outcomes are recorded. Hidden content remains available as review evidence, but not in normal views or AI input. Ordinary users cannot change audit records.

The backend checks permissions; database keys and constraints prevent broken links and duplicates. `docs/diagrams/assignment-2-data-model.mmd` shows the same core relationships.

2.7 Architecture Decision Records

ADR-01 — Modular Backend

Context: Sila's features share authorization and database transactions.

Options: Unstructured backend, modular backend, or microservices.

Decision: Use one FastAPI application divided into components C1–C7.

Consequences: Development, deployment, and testing remain manageable. Multiple WebSocket processes would require shared event distribution.

ADR-02 — Relational Database

Context: Records have relationships, and concurrent draft saves require transactions.

Options: In-memory storage, SQLite, PostgreSQL, or a document database.

Decision: Propose PostgreSQL for the planned application.

Consequences: Supports constraints and atomic updates; requires setup, migrations, and backups.

ADR-03 — Versioned Collaboration

Context: Conflicting saves must not silently overwrite newer content.

Options: Last-save-wins, version checks, or automatic merging.

Decision: Use atomic version checks and WebSocket broadcasts after saving.

Consequences: Prevents silent overwrites, but organizers must manually reconcile conflicts.

ADR-04 — Backend AI Integration

Context: Summarization must respect permissions and protect credentials.

Options: Browser calls, provider calls across multiple routes, or a dedicated backend adapter.

Decision: C7 handles input filtering, provider calls, response checks, and failures.

Consequences: Supports mocks and provider replacement. Accuracy is not guaranteed, and external processing requires university permission.

Project Scope

In Scope

- University email verification and authentication
- Student-created and official university communities
- Open and restricted community membership
- Rich-text posts, likes, and comments
- Post audience controls
- Real-time collaboration on event and announcement drafts
- Content reporting, moderation, and appeals
- On-demand AI discussion summarization

Out of Scope

- Public access for users outside the university
- Support for multiple universities
- Private direct messaging
- Polls and voting
- Native mobile application
- Advanced search
- Online shopping, advertising, or payments
- Academic grading and student assessment

These excluded features may be considered for future development, but they are not part of the current project scope.

---

3 Project Management and Team Collaboration

3.1 Team Structure

For Assignment 2, the Sila team will continue using the same division of responsibilities established during Assignment 1. Tasks were originally assigned by Fatma based on each member's strengths and comfort level. Everyone agreed to their responsibilities.

Team Member	Main Responsibilities

Fatma and Fatima	Maintain the requirements, user stories, acceptance criteria, and traceability documentation. Help verify that the application follows the agreed requirements.
Salama	Maintain the system architecture, diagrams, and design documentation. Check that the implementation remains consistent with the architecture.
Fatma	Coordinate project management, task allocation, progress tracking, and documentation of the development plan.
Mohammed	Lead the implementation and maintenance of the frontend, backend, database, and integration code.

Although each member has specific responsibilities, the team will remain flexible. If someone needs help with a task, any available team member can assist. This will help prevent delays and allow members to learn about different parts of the system.

Coordination and Knowledge Sharing

Microsoft Teams will remain the main communication platform for discussing tasks, asking questions, and sharing updates. Google Docs will be used for report collaboration, while GitHub will be used to share and review code changes. Canva will be used to maintain the architecture diagrams.

When a change affects both the code and the requirements or architecture, the responsible members will communicate before finalizing it. Team members will also review important updates so that everyone understands how the different parts of Sila work together.

3.2 Development Workflow

All code and documentation will be maintained in one shared GitHub repository. Assignment 2 will build on the workflow introduced during Assignment 1.

Branches and Pull Requests

The `main` branch will hold reviewed, working code. Each task will have its own short-lived branch, such as `feature/user-login` or `feature/community-posts`.

Changes will be submitted through pull requests (PRs) explaining what was changed, how it was tested, and whether AI tools were used.

Code Review

At least one teammate will review each pull request before it is merged. The reviewer will check that the changes match the requirements, follow the interface contracts, and pass the relevant tests.

During Assignment 1, PR #1 was merged before a teammate reviewed it. A review was requested afterward. For Assignment 2, the team plans to require teammate approval before merging.

Automated Checks

GitHub Actions will run automated tests when pull requests are submitted. The team plans to avoid merging changes when required tests fail.

Testing will gradually expand to cover the frontend, backend, and integration between components.

Issues and Task Tracking

The team proposes using GitHub Issues to organize Assignment 2 tasks. Each issue will describe the work and identify the responsible member.

A GitHub Projects board can track tasks using four stages: *To do*, *In progress*, *In review*, and *Done*.

Pull requests will be linked to their related issues so the team can follow development progress.

Recording Decisions and Requirement Changes

Interface contracts will be maintained in `docs/contracts/`, while important architecture decisions will be recorded through Architecture Decision Records (ADRs).

If a requirement or design decision changes, the team will review the change and update the relevant documentation to keep the report and implementation consistent.

Reviewing AI-Assisted Work

The team may use AI development tools, including Claude Code and Codex, to assist with coding, testing, and documentation. However, all AI-generated work must be reviewed by a team member.

The team will follow these steps:

1. Review and approve the proposed approach before implementing major changes.
2. Check that AI-generated code follows the approved requirements and interface contracts.
3. Run automated tests and manually test important features.

4. Make sure the responsible team member understands the code and can explain it independently.
5. Record AI assistance honestly in commits or pull request descriptions.
6. Require human review and avoid sharing passwords, API keys, or other secrets with AI tools.

These practices will help the team maintain code quality and understand the submitted application.

3.3 Development Approach

For Assignment 2, the Sila team plans to use an Agile approach with short development iterations. This approach allows the team to build the application gradually, test each feature, and make improvements based on feedback.

The team will prioritize the main features first, including university account verification, communities, posts, and audience controls. After these features work correctly, development will continue with likes, comments, real-time collaboration, moderation, and AI discussion summaries.

At the end of each iteration, the team will review the completed features and check whether they meet the functional requirements and acceptance criteria from Section 1.

Microsoft Teams will be used for communication, while GitHub will be used to manage development tasks and code changes. Testing and documentation will take place throughout development rather than being left until the final week.

This approach suits the team because it provides flexibility, allows members to support one another, and makes it easier to identify problems early.

3.4 Project Risks

The team identified several risks that could affect the development of Sila during Assignment 2.

Risk	Likelihood	Impact	Prevention or Response
Integration problems between the frontend and backend	Medium	High	Agree on API contracts early and test communication between components regularly.
Real-time collaboration may be difficult to implement	High	High	Develop and test WebSocket communication separately before connecting it to the full application.

Unauthorized access to private content	Medium	High	Apply backend permission checks and test restricted access.
AI-generated summaries may be inaccurate or unavailable	Medium	Medium	Check AI responses, keep original discussions accessible, and display errors when summarization fails.
Development delays	Medium	High	Track progress through GitHub and Microsoft Teams, prioritize essential features, and allow teammates to help when needed.
Code changes may introduce errors	Medium	Medium	Use pull requests, teammate reviews, and automated tests before merging changes.

The team will review these risks throughout development and adjust the plan if new problems appear.

3.5 Assignment 2 Timeline

The Sila team is currently in Week 8, and Assignment 2 is due in Week 14. The team plans to organize development across the remaining weeks, with each milestone producing features that can be demonstrated and tested.

Timeline	Milestone	Demonstrable Result
Week 8	Project setup and planning	Confirm the architecture, assign development tasks, prepare the React and FastAPI environments, and verify that the existing proof of concept runs.
Week 9	User accounts and communities	Students can register, log in, and create or join communities using test email verification.
Week 10	Posts and interactions	Students can create posts, select audiences, view permitted content, like posts, and add comments.
Week 11	Real-time collaboration	Authorized organizers can edit shared announcements, receive WebSocket updates, and detect conflicting changes.
Week 12	Moderation and AI integration	Moderators can review reports, users can submit appeals, and students can request AI-generated discussion summaries.

Week 13	Testing and improvements	The team tests permissions, errors, performance, and integration between components, while updating documentation.
Week 14	Final review and submission	The team demonstrates the working application, completes the report and repository, and submits Assignment 2.

Progress will be tracked through GitHub, with updates shared through Microsoft Teams. Testing and documentation will take place throughout development rather than only during the final week.

If delays occur, the team will focus on completing the required features first and postpone any optional improvements.

4 Proof of Concept

4.1 Purpose and scope

The PoC demonstrates a browser creating and retrieving posts through a Python backend. It covers part of FR-03 using fictional data. The code is in `poc/`; `poc/README.md` explains setup, execution, tests, and the demonstration. Section 2.4.1 describes the implemented contract.

4.2 Implementation

FastAPI serves one HTML page containing CSS and JavaScript. The page has a textbox, character counter, Publish button, and newest-first feed.

The browser sends `body` to `POST /posts`. The backend trims outer whitespace, checks the 1–2,000-code-point limit, assigns `post_id`, and stores the post in a Python list. Success returns `201`. `GET /posts` returns the list newest first with `200`. Invalid input returns `422` with the contract's exact message. Rejected requests consume no ID.

The backend decides whether text is valid. Both sides count Unicode code points; some visible emoji contain several. `textContent` displays HTML-like input as text, so it cannot become page markup.

Publish is disabled while saving. Success clears the input and reloads the feed. Validation failures preserve the input. A lost network response leaves the result uncertain; a successful save followed by a failed feed refresh has a separate message.

4.3 Verification

The recorded automated run passed 28 tests covering the contract's valid and invalid inputs, trimming, IDs, ordering, and the 2,000/2,001 boundaries, including emoji. GitHub Actions also passed on the branch, PR, and merged `main`.

These tests check the API and page response, not execution of browser JavaScript. PR #1 records separate browser checks. The README explains how to repeat the demonstration. No production performance or full accessibility result is claimed.

4.4 Simplifications and limitations

Planned system	**PoC**
React and rich text	HTML/CSS/JavaScript and plain text
PostgreSQL	Temporary Python list
Verified users and audience controls	No login or permissions
C1–C7 and WebSocket collaboration	One small backend using HTTP
Communities, moderation, and AI	Omitted

Browser refresh preserves posts while the backend runs. Restarting it clears posts and resets IDs. The PoC uses one process because separate processes would have separate lists.

The demo plan has a three-minute limit: show an empty feed; publish two posts; show trimming, newest-first order, and literal HTML; reject whitespace-only input; refresh; then restart to show data loss. A timed rehearsal has not been recorded.

4.5 Development record and AI assistance

Development used `feature/poc-posts`, with separate commits for the contract, code/tests, documentation, and CI. PR #1 was merged into `main` on 14 October 2026, preserving those commits.

Claude Code assisted with the plan, code, tests, and README. Codex assisted with repository setup, CI, workflow documentation, and the report. Commits and the PR identify AI assistance.